MCU – SoC – FPGA

Đánh đổi giữa hiệu năng, chi phí và độ linh hoạt

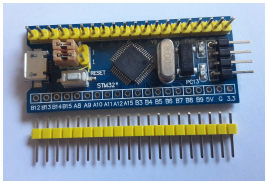
CO3053 Hệ thống nhúng · HK261 · Đề tài 09

Câu hỏi của bài

Khi nào nên chọn **MCU**, **SoC** hay **FPGA**?

- Xét theo 3 tiêu chí: **hiệu năng**, **chi phí**, **độ linh hoạt**
- *Không* nhằm chứng minh nền tảng nào "tốt nhất", vì đó là mục tiêu thiên vị mà `pe2` cảnh báo
- Mỗi con số trên slide có một nhãn cho biết nguồn gốc:
  [đo] [mô phỏng] [ước lượng công cụ] [tính] [nguồn]

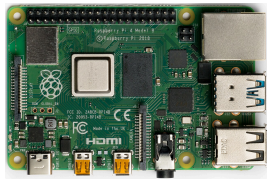
Ba kiến trúc



Popolon, CC BY-SA 4.0

MCU

Cortex-M, flash và SRAM trên chip
Không có MMU, nên không chạy Linux
Khởi động vài ms, ngắt dưới 1 µs



Laserlicht, CC BY-SA 4.0

SoC

Cortex-A có MMU và cache, cùng GPU, codec
Chạy **Linux**: driver, mạng có sẵn
Đổi lại: độ trễ khó đoán trước



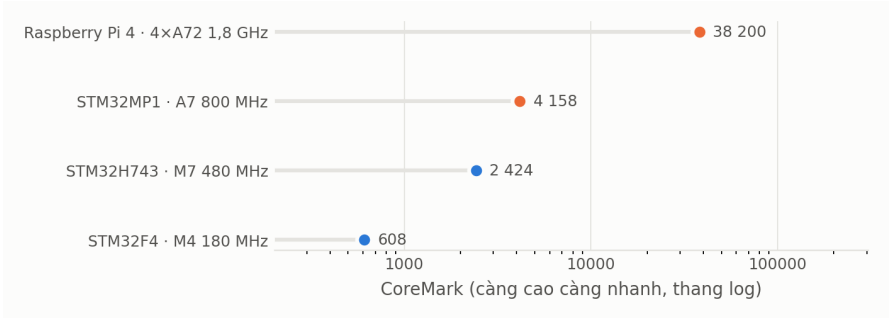
S03311251, CC BY-SA 4.0

FPGA

Mảng LUT, DSP, BRAM nối tùy ý
Mỗi phép nhân có một khối riêng: 1 mẫu/chu kỳ
Đổi lại: tự mô tả mọi thứ bằng HDL

Zynq (CPU và FPGA trên một chip) được xếp vào **phương án lai**

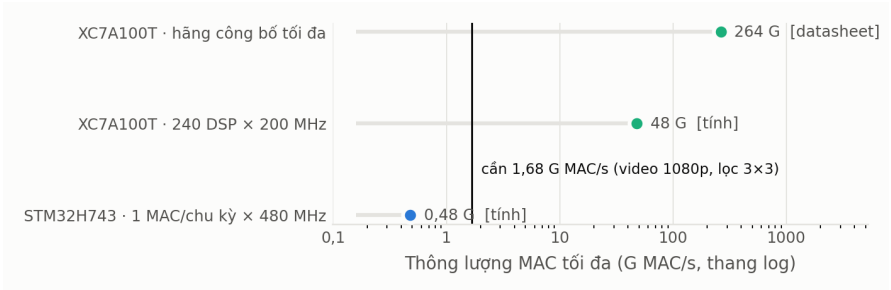
Hiệu năng CPU thuần: CoreMark



Dữ liệu: ST product overview; ST STM32H743; OpenBenchmarking (Pi 4, tổng 4 lõi). Nhóm vẽ lại.

SoC Pi 4 mạnh gấp ≈16× **MCU** H743 về CPU. CoreMark không áp dụng cho **FPGA**, vì FPGA không chạy chương trình tuần tự.

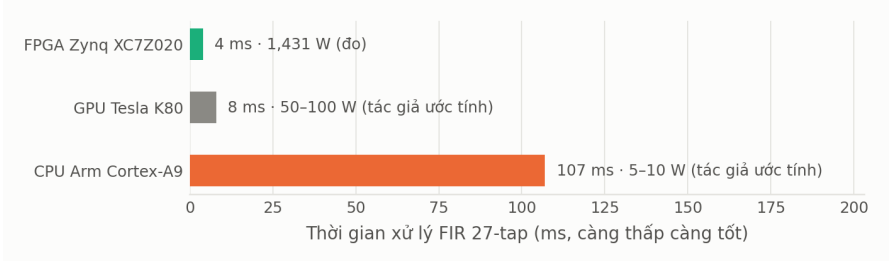
Thông lượng song song: bài lọc video 1080p



Dữ liệu: Lee & Seshia mục 8.1 (nhu cầu); datasheet XA Artix-7 (264 G). Giá trị [tính] theo giả thiết trong bài. Nhóm vẽ lại.

MCU chỉ đạt ≈29% nhu cầu dù hiệu suất 100%; **FPGA** chỉ dùng ≈3,5% năng lực [tính]

Bằng chứng đã công bố: FIR trên FPGA, GPU và CPU

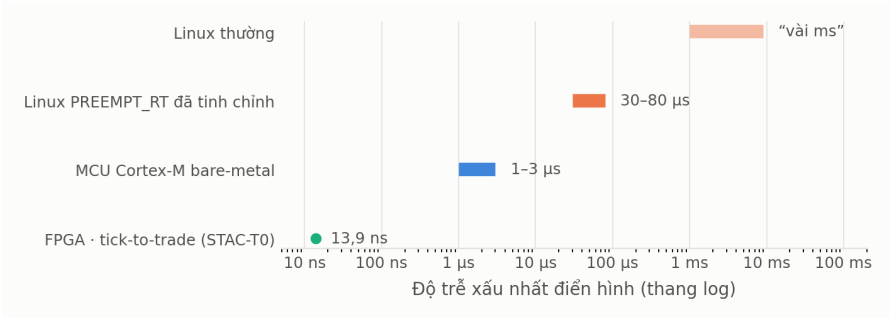


Dữ liệu: Arucu & Iliev, J. Low Power Electron. Appl. 15(3):40, 2025. Nhóm vẽ lại.

FPGA nhanh hơn Cortex-A9 ≈27× và tốn ít năng lượng hơn ≈95–190× [tính].

Công suất CPU và GPU là ước tính của tác giả; bài không có MCU.

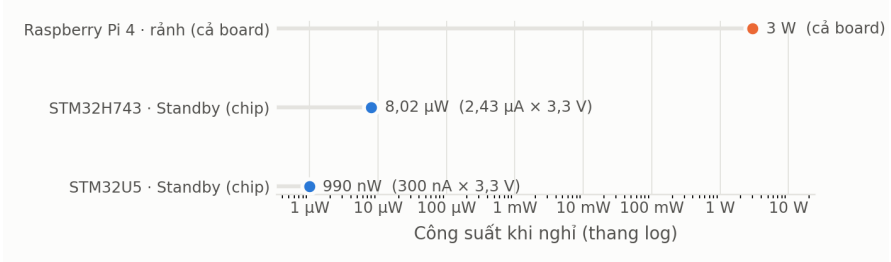
Độ trễ xấu nhất theo tài liệu



Dữ liệu: AC6 (MCU, Linux); A-Team Insight, STAC-T0 (FPGA). "Vài ms" không có số chính xác nên vẽ mờ. Nhóm vẽ lại.

Mỗi nền tảng cách nhau khoảng 1–2 bậc độ lớn. Các phép đo khác bản chất nhau, chỉ so bậc độ lớn.

Công suất khi nghỉ và giá



Dữ liệu: ST STM32H743; Arrow (STM32U5); TechFeed (Pi 4). Công suất MCU = dòng × 3,3 V [tính]. Nhóm vẽ lại.

Chênh ≈3 triệu lần: lý do **MCU** thống trị thiết bị chạy pin.

Giá: **MCU** £9–12/chip · **SoC** $35/board · **FPGA** €144/chip [nguồn; so chip với board]

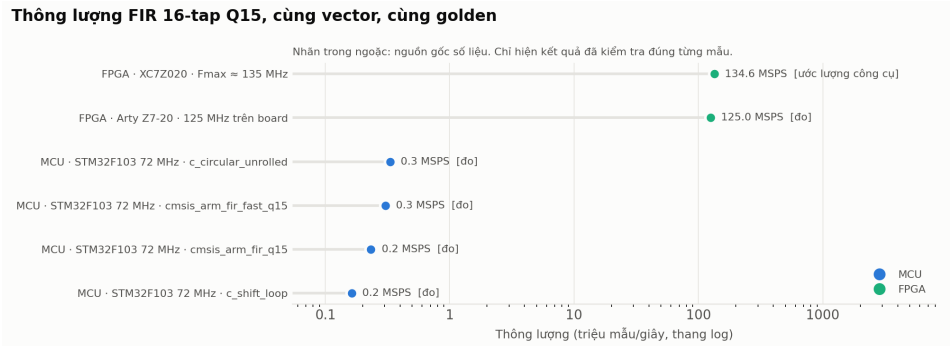
Thí nghiệm: một bài toán, ba nền tảng

FIR 16-tap Q15, 1024 mẫu: bậc thang, giá trị biên, mẫu ép bão hòa, mẫu ngẫu nhiên
Mọi nền tảng phải khớp **từng mẫu** với golden model viết bằng Python

MCU	STM32F103, Cortex-M3	4 phiên bản (C, CMSIS-DSP), đếm chu kỳ bằng DWT; độ trễ ngắt trong 3 kịch bản
SoC	Raspberry Pi 4	Cùng mã C, `-O2` và `-O3` (NEON); `cyclictest` khi rảnh và khi có tải
FPGA	Arty Z7-20 (XC7Z020 PL)	Pipeline 7 tầng; tự so với golden ngay trên chip ở 125 MHz; Vivado cho Fmax và công suất

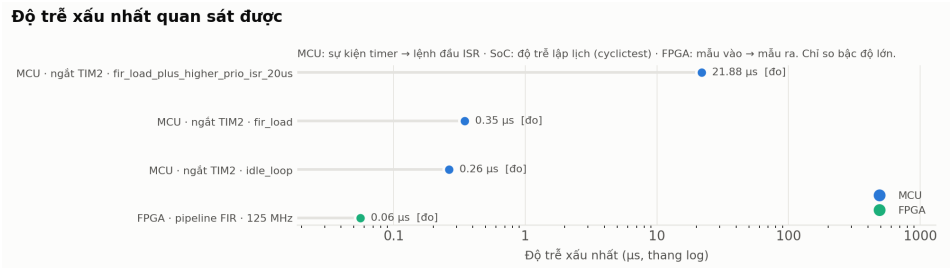
Cortex-M3 không có lệnh SIMD (SMLAD), nên CMSIS-DSP không được tăng tốc kiểu SIMD như trên M4

Kết quả: thông lượng



FPGA: 1,00 mẫu/chu kỳ, 125,0 MSPS [đo] **MCU**: 0,33 MSPS [đo] **SoC**: *chưa đo* MSPS [đo]

Kết quả: độ trễ xấu nhất



MCU ngắt khi chạy FIR: 0–347 ns · thêm ngắt ưu tiên cao hơn 20 µs: 21,9 µs [đo]
SoC Linux thường: *chưa đo* µs · PREEMPT_RT: *chưa đo* µs [đo] **FPGA** pipeline: 7 chu kỳ = 56 ns [đo]

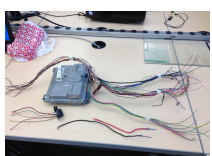
Kiểm chứng chéo với tài liệu

Câu hỏi	Nhóm đo được	Tài liệu nói
MCU đạt được bao nhiêu so với “1 MAC/chu kỳ”?	7%	PLAN giả định 100%
FPGA nhanh hơn CPU Cortex-A bao nhiêu lần?	*chưa đo*× (so với Pi 4)	≈27× (so với A9, Arucu & Iliev)
Độ trễ xấu nhất của Linux PREEMPT_RT	*chưa đo* µs	30–80 µs (AC6)
Độ trễ xấu nhất của Linux thường	*chưa đo* µs	vài ms (AC6)
Độ trễ ngắt MCU bare-metal	0–347 ns	1–3 µs (AC6)

Quan sát phụ: một lõi CPU x86 desktop lọc được khoảng 140 MSPS với bài 16-tap. Vậy FPGA không thắng nhờ tốc độ thô ở bài nhỏ, mà nhờ **thông lượng giữ nguyên khi tăng số tap**, **độ trễ cố định** và **công suất thấp**.

MCU: ECU ô tô

- Xe phổ thông có 25–35 MCU, xe sang 60–70
- AURIX TC3xx: 4/6 lõi chạy **lockstep**, nhắm tới **ASIL-D** (ISO 26262), dùng cho powertrain và inverter xe điện
- AEC-Q100 Grade 0: chạy được từ −**40 đến 150 °C**; NXP S32K14xW cam kết cung ứng 15 năm
- Khởi động vài ms, ngắt dưới 1 µs, giá vài USD



Snewkirk7953, CC BY-SA 3.0

Vì sao không chọn SoC: Linux khởi động mất vài giây, độ trễ không xác định

Vì sao không chọn FPGA: đắt; cấu hình nằm trong SRAM nên dễ bị lỗi bức xạ (SEU); khó chứng nhận cho hàng triệu xe

Ưu thế dừng ở: ADAS và giải trí dùng SoC; xu hướng domain controller gom nhiều ECU thành ít bộ hơn

Cũng chiếm ưu thế: điều khiển động cơ FOC (dưới 21 µs/50 µs), cảm biến chạy pin (300 nA, hơn 10 năm trên pin CR2032), đồ gia dụng

SoC: router Wi-Fi và điện thoại

- **Router:** Qualcomm IPQ = CPU chạy Linux/OpenWrt + Wi-Fi + switch + **khối xử lý gói tin cứng** (37,5 Mpps) + mã hóa
- **Điện thoại:** Snapdragon 8 Elite = CPU + GPU + NPU + ISP + modem 5G, chip 3 nm
- Linux lo điều khiển (control plane), khối cứng lo dữ liệu (data plane)



Raimond Spekking, CC BY-SA 4.0



Raimond Spekking, CC BY-SA 4.0

Cũng chiếm ưu thế: camera IP, TV và set-top box, gateway IoT, giải trí trên xe

Vì sao không chọn MCU: không có MMU nên không chạy Linux; phải tự viết lại TCP/IP stack

Vì sao không chọn FPGA: viết cả network stack và giao diện bằng HDL quá đắt cho thiết bị tiêu dùng

Ưu thế dừng ở: điều khiển thời gian thực cứng, nên trong máy vẫn có thêm MCU nhỏ

FPGA: dòng dữ liệu song song và tạo mẫu chip

"Tính toán song song" nói chung thì GPU phổ biến hơn. FPGA thắng khi dữ liệu **chảy liên tục từ ADC**, cần **độ trễ cố định**, và chuẩn còn thay đổi hoặc số lượng không đủ lớn để làm ASIC.

- **Siêu âm y tế:** beamforming 128–1024 kênh, 4,83 GS/s trên một FPGA; nối thẳng với front-end analog
- **Trạm 5G (radio unit):** low-PHY và DFE trên RFSoC hoặc Agilex; chuẩn O-RAN còn đổi nên cần nạp lại được
- **Tạo mẫu ASIC:** HAPS ghép nhiều FPGA chạy RTL tới 1,6 tỉ cổng trước khi đúc chip
- **HFT:** tick-to-trade 13,9 ns



Erereony, CC BY 4.0

Vì sao không chọn MCU: thiếu ≈3,5× năng lực dù đạt hiệu suất 100%

Vì sao không chọn SoC: có thể đủ thông lượng nhờ SIMD/GPU, nhưng mất tính xác định và tốn điện

Ưu thế dừng ở: số lượng lớn và thuật toán ổn định thì ASIC thắng (ISP, modem trong điện thoại)

Phương án lai CPU + FPGA: ADAS và điều khiển công nghiệp

- Bài toán cần **cả** hệ điều hành phong phú **lẫn** timing hoặc xử lý song song cứng
- Zynq UltraScale+ gom trên một chip: 4×A53 chạy Linux, 2×R5 lockstep chạy real-time, và fabric FPGA
- Lecture 3, Option #2: CPU lo tính toán mềm dẻo, FPGA giữ timing actuator kể cả khi CPU bận
- FIR trên SoC FPGA+ARM nhanh gấp 8,86× ARM đơn, giá/hiệu năng tốt nhất trong các cấu hình được thử [nguồn]



S03311251, CC BY-SA 4.0

Zynq UltraScale+: Arm + FPGA trên một chip

Cái giá: chi phí cao hơn và linh hoạt thấp hơn so với một CPU đơn

Tóm tắt các lĩnh vực đặc trưng

	Lĩnh vực đặc trưng	Yếu tố quyết định	Vì sao không chọn hai nền tảng kia
MCU	ECU ô tô, động cơ, cảm biến chạy pin	ASIL-D, 150 °C, dòng nghỉ cỡ nA, vài USD	SoC chậm khởi động và không xác định; FPGA đắt
SoC	Router Wi-Fi, điện thoại, camera IP	Linux + khối tăng tốc cứng	MCU không có MMU; FPGA tốn công HDL
FPGA	Siêu âm, 5G radio, tạo mẫu ASIC, HFT	Song song + I/O trực tiếp + tái cấu hình	MCU thiếu sức; SoC không xác định
Lai	ADAS, công nghiệp	OS + timing cứng	Một nền tảng đơn không đủ

Ma trận quyết định

Nếu bài toán…	Chọn	Vì
Chạy pin nhiều năm, tác vụ nhẹ	**MCU**	300 nA ở Standby
Deadline vài chục µs, cần chứng nhận an toàn	**MCU**	Ngắt dưới 1 µs; ASIL-D
Cần Linux, camera, mạng; timing mềm	**SoC**	CoreMark gấp ≈16×; driver có sẵn
Trên ≈0,5 G MAC/s liên tục, I/O tốc độ cao	**FPGA** / lai	1 mẫu/chu kỳ, ≈44–48 G MAC/s
Độ trễ cỡ ns	**FPGA**	13,9 ns
Cần cả OS lẫn timing cứng	Lai	Zynq, Lecture 3

Muốn timing và thông lượng cao hơn thì phải trả bằng **giá cao hơn** và **linh hoạt thấp hơn**

Hạn chế

- Số liệu tài liệu đến từ nhiều nguồn với điều kiện đo khác nhau
- Thí nghiệm của nhóm chỉ có **một** bài toán (FIR 16-tap); ba board ở ba phân khúc giá khác nhau
- Công suất FPGA là **ước lượng của Vivado**, không phải số đo
- Độ trễ ở ba nền tảng là ba đại lượng khác nhau (ngắt, lập lịch, pipeline)
- Số liệu thị phần đến từ các công ty nghiên cứu thị trường và có thể lệch nhau

Không có nền tảng tốt nhất

chỉ có nền tảng **đủ** cho con số quyết định của bài toán,
với chi phí thấp nhất

Hỏi đáp

Phụ lục: chuẩn bị cho câu hỏi

- **“MCU đã tối ưu chưa?”** Có bản CMSIS-DSP. M3 không có SIMD, bản tốt nhất đạt 216,3 chu kỳ/mẫu.
- **“Jitter của FPGA bằng 0 thật à?”** Mô phỏng RTL không có thời gian thật. Trên chip, độ trễ cố định theo số chu kỳ, chỉ còn clock jitter cỡ ps.
- **“Sao so chip với board?”** Chỉ để so bậc độ lớn, và nhóm đã ghi rõ trên từng bảng.
- **“Công suất FPGA đo bằng gì?”** Không đo; đó là Vivado `report_power` (vector-less) [ước lượng công cụ].
- **“Sao không có NRE?”** Chưa có số công bố. Nhóm dùng thời gian thực tế viết bản C và bản HDL làm dẫn chứng định tính.
- **“Tăng lên 64 tap thì sao?”** MCU chậm đi khoảng 4×. FPGA vẫn 1 mẫu/chu kỳ, chỉ tốn thêm DSP và vài tầng pipeline.

Nguồn chính

- ST: STM32H7/F4/MP1 overview, STM32H743; OpenBenchmarking: Raspberry Pi 4
- Arucu & Iliev, J. Low Power Electron. Appl., 2025 (FIR trên FPGA, GPU, CPU)
- AC6: độ trễ Linux PREEMPT_RT; A-Team Insight: Exegy/AMD STAC-T0
- Embedded World 02/2026 (số MCU mỗi xe); Automotive World (AURIX TC3xx); Future Electronics (NXP S32K Grade 0)
- Macnica, CNX Software (Qualcomm IPQ); GSMArena (Snapdragon 8 Elite)
- Kou et al., 2023 (beamforming siêu âm trên FPGA); HPCwire (Fujitsu + Xilinx O-RAN); Intel (Agilex O-RU); Synopsys HAPS
- eeNews Europe (Zynq UltraScale+); Lecture 2, Lecture 3; Lee & Seshia mục 8.1

Danh sách đầy đủ kèm đường link nằm trong báo cáo (`report.pdf`).

Nguồn hình ảnh

Ảnh chụp: Wikimedia Commons, giữ nguyên, chỉ thu nhỏ. Biểu đồ ghi “Nhóm vẽ lại” được dựng từ số liệu đã công bố, không chép hình gốc.

- Core Learning Board module Arduino STM32 F103 C8T6.jpg — Popolon, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Core_Learning_Board_module_Arduino_STM32_F103_C8T6.jpg
- Raspberry Pi 4 Model B - Top.jpg — Laserlicht, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Raspberry_Pi_4_Model_B_-_Top.jpg
- Xilinx Artix7 XC7A100T FGG484 on PCB.jpg — S03311251, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Xilinx_Artix7_XC7A100T_FGG484_on_PCB.jpg
- FPGA cell example.png — Petter.kallstrom, Public domain. https://commons.wikimedia.org/wiki/File:FPGA_cell_example.png
- ECU and wire bundles.JPG — Snewkirk7953, CC BY-SA 3.0. https://commons.wikimedia.org/wiki/File:ECU_and_wire_bundles.JPG
- Intel FWIXP422BB on mainboard of UMTS Router Surf@home II, o2-8338.jpg — Raimond Spekking, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Intel_FWIXP422BB_on_mainboard_of_UMTS_Router_Surf@home_II,_o2-8338.jpg
- HTC Desire - main board - Qualcomm QSD8250-2.jpg — Raimond Spekking, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:HTC_Desire_-_main_board_-_Qualcomm_QSD8250-2.jpg
- Sonoscape Ultrasound Machine.jpg — Erereony, CC BY 4.0.
  https://commons.wikimedia.org/wiki/File:Sonoscape_Ultrasound_Machine.jpg
- Xilinx ZynqUltrascale XCZU4CG on PCB.jpg — S03311251, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Xilinx_ZynqUltrascale_XCZU4CG_on_PCB.jpg